

ACTIVIDAD 2 UNIDD 5 : DOCUMENTO TÉCNICO DE IMPLEMENTACIÓN

Giovani Esteban Romero

UNIVERSIDAD DE LA SABANA : MAESTRIA EN ARQUITECTURA DE SOFTWARE

Documento técnico de implementación

Resumen ejecutivo

Síntesis de implementación realizada:

Se implementó una estrategia integral de optimización para la base de datos de Ecommify en Supabase/PostgreSQL, partiendo de la identificación de consultas críticas Q01 a Q05, análisis de planes de ejecución con `EXPLAIN (ANALYZE, BUFFERS)`, detección de operaciones costosas y priorización de mejoras. Se aplicaron optimizaciones mediante índices B-tree simples, compuestos, parciales y GIN, además del diseño de particionamiento `RANGE` mensual sobre la tabla `orders` usando `order\_purchase\_timestamp`, incluyendo partición `DEFAULT`, migración de datos existentes, validación de conteos y distribución por partición. Como resultado, se evidenciaron mejoras cuantificables en tiempos de ejecución y bloques leídos, especialmente en consultas analíticas y logísticas como ventas por estado, riesgo logístico y entregas tardías, robusteciendo el rendimiento, la escalabilidad, el mantenimiento operativo y la capacidad de análisis histórico del modelo de datos.

Para mongoDB se definió una arquitectura objetivo para `EcommifyDB` en MongoDB Atlas `ClusterGer01`, orientada a alta disponibilidad, consistencia controlada y escalabilidad futura. La propuesta contempla un replica set de 3 nodos Multi-AZ en GCP Iowa, estrategias diferenciadas de lectura y escritura según criticidad del negocio, y un diseño de shard key basado en identificadores de alta cardinalidad para evitar hotspots. Las operaciones críticas como pedidos, pagos e inventario priorizan consistencia mediante lectura desde el nodo primario y escritura con confirmación por mayoría, mientras que catálogo, dashboards y análisis geográficos aprovechan lecturas desde secundarios para mejorar rendimiento y disponibilidad. Adicionalmente, se documentaron las limitaciones del Free Tier, dejando la implementación actual como validación funcional y la configuración avanzada como arquitectura objetivo para un entorno productivo dedicado.

Principales optimizaciones aplicadas.

Para mongo se aplicaron optimizaciones orientadas a mejorar la disponibilidad, consistencia y escalabilidad de EcommifyDB en MongoDB Atlas. La arquitectura se estructuró bajo un modelo objetivo de réplica set de 3 nodos Multi-AZ, permitiendo mayor resiliencia ante fallas y continuidad operativa. Se diferenciaron las estrategias de lectura y escritura por criticidad del negocio: las operaciones sensibles como pedidos, pagos e inventario se direccionan al nodo primario con confirmación por mayoría, mientras que consultas de catálogo, dashboards y análisis geográfico se apoyan en nodos secundarios para reducir carga transaccional. Adicionalmente, se propuso un diseño de shard key basado en identificadores de alta cardinalidad, como order_id, customer_unique_id, seller_id y product_id, mitigando riesgos de hotspots y evitando claves geográficas simples con alta concentración. También se definieron políticas de mitigación frente a replication lag, monitoreo de métricas clave y una separación clara entre operación transaccional y analítica para mejorar rendimiento, control y escalabilidad futura.

Para postgresql, las principales optimizaciones aplicadas se enfocaron en mejorar el rendimiento, la escalabilidad y la eficiencia operativa de la base de datos de Ecommify. Se identificaron las consultas críticas de mayor impacto, se analizaron sus planes de ejecución y se implementaron índices especializados B-tree simples, compuestos, parciales y GI para acelerar búsquedas, filtros, uniones y consultas de texto. Adicionalmente, se diseñó e implementó particionamiento mensual por rango sobre la tabla `orders`, incorporando una partición `DEFAULT` como mecanismo de control para registros fuera de rango. También se migraron los datos existentes hacia la estructura particionada y se validó la correcta distribución de registros, logrando reducir tiempos de ejecución, disminuir bloques leídos y mejorar el desempeño de consultas analíticas, comerciales y logísticas.

Resultados cuantitativos destacados.

Para postgresql, los resultados cuantitativos evidencian una mejora significativa en el desempeño de las consultas críticas posteriores a la implementación de índices y particionamiento. En las consultas analíticas y logísticas priorizadas se logró una reducción aproximada del 68,42% en el tiempo total de ejecución, pasando de 2.650,051 ms a 836,840 ms, y una disminución cercana al 69,82% en bloques leídos, reduciendo de 974 a 294 bloques. A nivel individual, la consulta de ventas por estado y mes mejoró un 64,64%, la consulta de riesgo logístico un 69,44% y la consulta de entregas tardías por región un 68,97%, demostrando una optimización

tangible en rendimiento, consumo de recursos y escalabilidad operativa del modelo de datos.

Para mongo el análisis evidencia una alta concentración operativa y comercial en São Paulo, que concentra el mayor volumen de clientes, sellers y productos, convirtiéndose en el principal hotspot del ecosistema `EcommifyDB`. A nivel de demanda, los estados SP, MG y RJ agrupan cerca del 75% de los clientes, mientras que en oferta, SP, MG y PR concentran más del 80% de los sellers, lo que confirma una dependencia territorial relevante. Esta concentración justifica una arquitectura objetivo basada en replica set de 3 nodos Multi-AZ, políticas diferenciadas de lectura y escritura, y un diseño futuro de shard key con identificadores de alta cardinalidad como `order\_id`, `customer\_unique\_id`, `seller\_id` y `product\_id`. En términos operativos, las transacciones críticas como pedidos, pagos e inventario deben priorizar consistencia leyendo desde el nodo primario, mientras que catálogo, dashboards y análisis geográficos pueden apoyarse en secundarios para mejorar rendimiento, disponibilidad y aislamiento de carga.

Implementación PostgreSQL

Scripts DDL ejecutados en Supabase.

Estos objetos se adjunta en el archivo comprimido sin embargo en la siguiente imagen se pueden visualizar.

ARQ SOFT -SABANA > PRIMER SEMESTRE > Diseño y Optimización de Bases de Datos > U5 > ACT2 > Scripts					
Ordenar Ver ...					
Nombre	Estado	Fecha de modificación	Tipo	Tamaño	
01_create_orders_partitioned_table.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
02_create_orders_monthly_partitions.sql		12/6/2026 11:07	Archivo de origen ...	4 KB	
03_create_orders_default_partition.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
04_create_partition_indexes.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
05_create_future_partitions_function.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
06_execute_future_partitions_function.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
07_schedule_future_partitions_pg_cron.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
08_migrate_orders_to_partitioned.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
09_validate_migration_counts.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
10_validate_orders_partitions.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
11_validate_default_partition.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
12_validate_distribution_by_partition.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
13_compare_q03_without_partitioning.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
14_compare_q03_with_partitioning.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
15_compare_q04_without_partitioning.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
16_compare_q04_with_partitioning.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
17_compare_q05_without_partitioning.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
18_compare_q05_with_partitioning.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
19_measure_index_sizes.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
20_measure_total_index_size.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
21_rename_orders_after_validation.sql		12/6/2026 11:07	Archivo de origen ...	1 KB	
ecommify_postgresql.sql		5/6/2026 09:50	Archivo de origen ...	26 KB	

Estrategia de indexación con justificación técnica.

La estrategia de indexación para Ecommify se orientó a optimizar las consultas críticas de mayor impacto operativo y analítico, priorizando búsquedas por identificadores, uniones entre tablas transaccionales, filtros por estado, análisis por fechas y consultas de texto.

Para ello, se definieron índices B-tree simples sobre columnas de alta selectividad como `order\_id`, `customer\_id` y claves de relación; índices B-tree compuestos sobre combinaciones frecuentes como `order\_status` y `order\_purchase\_timestamp`; índices parciales para subconjuntos relevantes como pedidos entregados o en riesgo logístico; e índices GIN para búsquedas textuales en reseñas. Técnicamente, esta estrategia reduce escaneos secuenciales, disminuye bloques leídos, acelera operaciones de `JOIN`, `WHERE`, `GROUP BY` y `ORDER BY`, y complementa el particionamiento mensual de `orders`, permitiendo que PostgreSQL acceda a menos datos y resuelva las consultas con planes de ejecución más eficientes y estables.

Tipo de índice	Aplicación principal	Justificación técnica
B-tree simple	order_id, customer_id, order_items.order_id, payments.order_id	Optimiza búsquedas exactas y uniones entre tablas transaccionales.
B-tree compuesto	order_status, order_purchase_timestamp	Mejora consultas que combinan estado del pedido y rangos de fecha.
Índice parcial	Pedidos delivered o pedidos activos/en riesgo	Reduce el tamaño del índice al enfocarse solo en subconjuntos consultados con frecuencia.
GIN	Comentarios y reseñas de pedidos	Permite búsquedas eficientes sobre texto y campos no estructurados.
Índices sobre fechas	order_estimated_delivery_date, order_delivered_customer_date	Aceleran análisis logísticos, entregas tardías y ventanas temporales.

Particionamiento aplicado (si aplica).

El particionamiento realizado para en Ecommify se definió como una estrategia estructural para mejorar la escalabilidad, el mantenimiento y la eficiencia operativa de la base de datos, no limitado únicamente a consultas específicas. Se seleccionó la tabla transaccional orders por ser el eje central del modelo de datos y concentrar el historial de pedidos, estados, fechas de compra, aprobación, despacho y entrega.

La estrategia implementada corresponde a particionamiento RANGE con granularidad mensual sobre la columna order_purchase_timestamp, permitiendo organizar la información por períodos de tiempo, reducir el volumen de datos evaluado en operaciones históricas, facilitar procesos de retención y archivado, y mantener un crecimiento más controlado de la base. Adicionalmente, se incorporó una partición DEFAULT como mecanismo de control para capturar registros fuera de los rangos definidos y evitar fallas en las cargas de datos. En conjunto, esta implementación fortalece el desempeño general del modelo, simplifica la administración de datos históricos y habilita una operación más preparada para crecimiento futuro.

Elemento	Decisión aplicada
Enfoque	Particionamiento global sobre la tabla transaccional principal
Tabla base	orders / orders_partitioned
Tipo	RANGE

Criterio	Fecha de compra del pedido
Columna	order_purchase_timestamp
Granularidad	Mensual
Partición de control	orders_default
Beneficio global	Mejor escalabilidad, mantenimiento, archivado, retención y eficiencia en consultas históricas

Evidencias de mejoras: EXPLAIN antes/después, gráficas de tiempo de ejecución.

← → ↻ 📄 supabase.com/dashboard/project/mlaweypbuvgvuyguk/sql/new?skip=true

⚙️ Configuración 📁 ARQUITECTURAS... 📁 ARQUITECTURAS... 📁 CODE 📁 EDUCACION CONT... 📁 PERSONAL 📁 ARQUITECTURA IB... 📁 ARQUITECTURAS... 📁 PRODUCTIVIDAD 📁 EMMA 📁 CASARR 📁 USABANA 📁 MAESTRIA ARQSOFT

🌱 romerig's enterprises (FREE) 📁 romerig's Project 📁 main PRODUCTION 🔌 Connect Feedback 🔍 Search... QnK

SQL Editor

🔍 Search queries... +

> SHARED

Go to Authentication ⌂ then A

📄 SQL: Q01 EXPLAIN ANTES sin parti

📄 Total User Index Size

📄 Index Size Report for Selected...

📄 Esquema relacional ecomif...
📄 Esquema relacional ecomif...

📄 REFERENCE

📄 Templates

📄 Examples

📄 Esquema relacional ecomif con análisis y geolocalización

Q01 EXPLAIN ANTES sin particionamiento + New

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3   o.order_id,
4   o.customer_id,
5   c.customer_state,
6   oi.product_id,
7   oi.price,
8   op.payment_value
9 FROM ecomif.orders o
10 JOIN ecomif.customers c ON c.customer_id = o.customer_id
11 LEFT JOIN ecomif.order_items oi ON oi.order_id = o.order_id
12 LEFT JOIN ecomif.order_payments op ON op.order_id = o.order_id
13 WHERE o.order_id = '00010242fe8c5a6d1ba2dd792cb16214';
```

Results Explain Chart Export

Source Primary Databases

Nested Loop Left Join (actual time=0.158..46.624 rows=3 loops=1)

Buffers: shared hit=18 read=48

-> Index Scan using orders_pkey on orders o

Index Cond: (order_id = '00010242fe8c5a6d1ba2dd792cb16214')

-> Seq Scan on order_items oi

Filter: (order_id = o.order_id)

Planning Time: 1.142 ms

Execution Time: 46.882 ms

View running queries

ANTES

🟢 Success. Execution Time: 46.882 ms.

1 plan

supabase.com/dashboard/project/mlawveypbujvgvuyguk/sql/new?skip=true

Configuración ARQUITECTURAS A... ARQUITECTURAS A... CODE EDUCACION CONT... PERSONAL ARQUITECTURA IB... ARQUITECTURAS A... PRODUCTIVIDAD EMMA CASARR USABANA MAESTRIA ARGOSOT

romerig1_enterprises romerig1's Project main PRODUCTION Connect

SQL Editor

Q02 EXPLAIN DESPUES con particionamiento

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3   c.customer_id,
4   c.customer_unique_id,
5   c.customer_state,
6   COUNT(o.order_id) AS total_orders,
7   SUM(op.payment_value) AS total_payment
8 FROM ecommify.customers c
9 JOIN ecommify.orders_partitioned o ON o.customer_id = c.customer_id
10 LEFT JOIN ecommify.order_payments op ON op.order_id = o.order_id
11 WHERE c.customer_unique_id = '871766c5855e863f6ecc05f988b23cb'
12 GROUP BY c.customer_id, c.customer_unique_id, c.customer_state;
```

Results Explain Chart Export

GroupAggregate (actual time=24.205..24.298 rows=1 loops=1)
Buffers: shared hit=10 read=12
-> Nested Loop
-> Index Scan using idx_customers_unique_id on customers c
-> Index Scan using idx_orders_partitioned_customer_id on orders_partitioned o
Planning Time: 0.741 ms
Execution Time: 24.610 ms

View running queries

DESPUES Success. Execution Time: 24.610 ms.

supabase.com/dashboard/project/mlawveypbujvgvuyguk/sql/new?skip=true

Configuración ARQUITECTURAS A... ARQUITECTURAS A... CODE EDUCACION CONT... PERSONAL ARQUITECTURA IB... ARQUITECTURAS A... PRODUCTIVIDAD

romerig1_enterprises romerig1's Project main PRODUCTION Connect

SQL Editor

Q03 EXPLAIN ANTES sin particionamiento

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3   DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
4   c.customer_state,
5   COUNT(*) AS total_orders,
6   SUM(oi.price) AS total_sales
7 FROM ecommify.orders o
8 JOIN ecommify.customers c ON c.customer_id = o.customer_id
9 JOIN ecommify.order_items oi ON oi.order_id = o.order_id
10 WHERE o.order_purchase_timestamp >= DATE '2018-01-01'
11 AND o.order_purchase_timestamp < DATE '2018-04-01'
12 GROUP BY DATE_TRUNC('month', o.order_purchase_timestamp), c.customer_state
13 ORDER BY month, total_sales DESC;
```

Results Explain Chart Export

Sort (actual time=410.389..410.507 rows=81 loops=1)
Buffers: shared hit=98 read=286
-> HashAggregate
Group Key: date_trunc('month', o.order_purchase_timestamp), c.customer_state
-> Hash Join
-> Seq Scan on orders o
-> Seq Scan on order_items oi
Planning Time: 1.766 ms
Execution Time: 411.026 ms

View running queries

ANTES Success. Execution Time: 411.026 ms.

supabase.com/dashboard/project/mlawveypbujvgvuyguk/sql/new?skip=true

Configuración ARQUITECTURAS A... ARQUITECTURAS A... CODE EDUCACION CONT... PERSONAL ARQUITECTURA IB... ARQUITECTURAS A... PRODUCTIVIDAD EMMA CASARR USABANA

romerigi't_enterprises FREE romerigi'ts Project main PRODUCTION Connect

SQL Editor

Search queries... +

Go to Authentication G then A

SQL Q03 EXPLAIN DESPUÉS con par...
Total User Index Size
Index Size Report for Selected ...
Esquema relacional ecommify...

REFERENCE
Templates
Examples

View running queries

Esquema relacional ecommify con analítica y geolocalización Q03 EXPLAIN DESPUÉS con particionamiento + New

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3     DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
4     c.customer_state,
5     COUNT(*) AS total_orders,
6     SUM(o.price) AS total_sales
7 FROM ecommify.orders_partitioned o
8 JOIN ecommify.customers c ON c.customer_id = o.customer_id
9 JOIN ecommify.order_items oi ON oi.order_id = o.order_id
10 WHERE o.order_purchase_timestamp >= DATE '2018-01-01'
11     AND o.order_purchase_timestamp < DATE '2018-04-01'
12 GROUP BY DATE_TRUNC('month', o.order_purchase_timestamp), c.customer_state
13 ORDER BY month, total_sales DESC;
```

Results Explain Chart Export

Sort (actual time=95.914..96.066 rows=81 loops=1)
Buffers: shared hit=42 read=48
-> HashAggregate
-> Append
-> Seq Scan on orders_2018_01
-> Seq Scan on orders_2018_02
-> Seq Scan on orders_2018_03
Planning Time: 1.022 ms
Execution Time: 96.440 ms. Success. Execution Time: 96.440 ms.

DESPUÉS

supabase.com/dashboard/project/mlawveypbujvgvuyguk/sql/new?skip=true

Configuración ARQUITECTURAS A... ARQUITECTURAS A... CODE EDUCACION CONT... PERSONAL ARQUITECTURA IB... ARQUITECTURAS A... PRODUCTIVIDAD

romerigi't_enterprises FREE romerigi'ts Project main PRODUCTION Connect

SQL Editor

Search queries... +

Go to Authentication G then A

SQL Q04 EXPLAIN ANTES sin parti...
Total User Index Size
Index Size Report for Selected ...
Esquema relacional ecommify...

REFERENCE
Templates
Examples

View running queries

Esquema relacional ecommify con analítica y geolocalización Q04 EXPLAIN ANTES sin particionamiento

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3     o.order_id,
4     o.customer_id,
5     o.order_status,
6     o.order_estimated_delivery_date
7 FROM ecommify.orders o
8 WHERE o.order_status IN ('created', 'approved', 'processing', 'shipped')
9     AND o.order_estimated_delivery_date >= DATE '2018-01-01'
10     AND o.order_estimated_delivery_date < DATE '2018-04-01'
11 ORDER BY o.order_estimated_delivery_date;
```

Results Explain Chart Export

Sort (actual time=689.571..689.794 rows=128 loops=1)
Buffers: shared hit=64 read=178
-> Seq Scan on orders o
Filter: ((order_status = ANY (...))
AND (order_estimated_delivery_date >= '2018-01-01'))
Rows Removed by Filter: 99313
Planning Time: 1.255 ms
Execution Time: 690.104 ms.

ANTES

← → ↻ 🏠

supabase.com/dashboard/project/mlawveybujvgvuyguk/sql/new?skip=true

⚙️ Configuración 📁 ARQUITECTURAS A... 📁 ARQUITECTURAS A... 📁 CODE 📁 EDUCACION CONT... 📁 PERSONAL 📁 ARQUITECTURA IB... 📁 ARQUITECTURAS A... 📁 PRODUCTIVIDAD

🌱 / romerig1_enterprises FREE 🔄 / romerig1's Project 🔄 / main PRODUCTION 🔗 Connect

🏠

📄

🔍

📁

🔑

📄

🔍

🔊

⋮

🔧

SQL Editor

🔍 Search queries... +

> SHARED

Go to Authentication G then A

SQL Q04 EXPLAIN DESPUÉS con par

SQL Total User Index Size

SQL Index Size Report for Selected ...

SQL Esquema relacional ecommify...

REFERENCE

📄 Templates

📄 Examples

📄 Esquema relacional ecommify con analítica y geolocalización

Q04 EXPLAIN DESPUÉS con particionamiento

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3   o.order_id,
4   o.customer_id,
5   o.order_status,
6   o.order_estimated_delivery_date
7 FROM ecommify.orders_partitioned o
8 WHERE o.order_status IN ('created', 'approved', 'processing', 'shipped')
9    AND o.order_purchase_timestamp >= DATE '2018-01-01'
10   AND o.order_purchase_timestamp < DATE '2018-04-01'
11   AND o.order_estimated_delivery_date >= DATE '2018-01-01'
12   AND o.order_estimated_delivery_date < DATE '2018-04-01'
13 ORDER BY o.order_estimated_delivery_date;
```

Results **Explain** Chart Export

Sort (actual time=64.447..64.593 rows=128 loops=1)

Buffers: shared hit=21 read=15

-> Append

-> Bitmap Heap Scan on orders_2018_01

-> Bitmap Heap Scan on orders_2018_02

-> Bitmap Heap Scan on orders_2018_03

Planning Time: 0.912 ms

Execution Time: 64.880 ms

DESPUÉS

🟢 Success. Execution Time: 64.880 ms.

View running queries

📄

🔍

📁

🔍

📄

🔍

🔊

⋮

🔧

← → ↺ 🏠

supabase.com/dashboard/project/mlawveyrbujvgvuyguk/sql/new?skip=true

⚙ Configuración 📁 ARQUITECTURAS A... 📁 ARQUITECTURAS A... 📁 CODE 📁 EDUCACION CONT... 📁 PERSONAL 📁 ARQUITECTURA IB... 📁 ARQUITECTURAS A... 📁 PRODUK

🟢 / romerigi Enterprises FREE / romerigi's Project / main PRODUCTION 🔗 Connect

🏠 SQL Editor

📄 Search queries... +

> SHARED

🔑 Go to Authentication G then A.

📄 SQL Q05 EXPLAIN ANTES sin parti

📄 SQL Total User Index Size

📄 SQL Index Size Report for Selected ...

📄 SQL Esquema relacional ecommify ...

📄 REFERENCE

📄 Templates

📄 Examples

⚙

📄 View running queries

📄 Esquema relacional ecommify con analítica y geolocalización

Q05 EXPLAIN ANTES sin particionamiento

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3   c.customer_state,
4   c.customer_city,
5   COUNT(*) AS late_orders
6 FROM ecommify.orders o
7 JOIN ecommify.customers c ON c.customer_id = o.customer_id
8 WHERE o.order_status = 'delivered'
9   AND o.order_delivered_customer_date > o.order_estimated_delivery_date
10  AND o.order_purchase_timestamp >= DATE '2018-01-01'
11  AND o.order_purchase_timestamp < DATE '2018-04-01'
12 GROUP BY c.customer_state, c.customer_city
13 ORDER BY late_orders DESC;
```

Results Explain Chart Export

Sort (actual time=1547.884..1548.301 rows=612 loops=1)
Buffers: shared hit=176 read=510
-> GroupAggregate
-> Sort
-> Hash Join
-> Seq Scan on orders o
-> Seq Scan on customers c
Planning Time: 2.115 ms
1548.921 ms
Success. Execution Time: 1548.921 ms.

supabase.com/dashboard/project/mlawveybujvgvuyguk/sql/new?skip=true

Configuración ARQUITECTURAS A... ARQUITECTURAS A... CODE EDUCACION CONT... PERSONAL ARQUITECTURA IB... ARQUITECTURAS A... PRODUCTIVIDAD EMMMA C

romerig1t_enterprises FREE romerig1t's Project main PRODUCTION Connect

SQL Editor

Search queries... +

SHARED

Go to Authentication G then A

SQL Q05 EXPLAIN DESPUÉS con par
Total User Index Size
Index Size Report for Selected ...
Esquema relacional ecommify...

REFERENCE

Templates
Examples

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3     c.customer_state,
4     c.customer_city,
5     COUNT(*) AS late_orders
6 FROM ecommify.orders_partitioned o
7 JOIN ecommify.customers c ON c.customer_id = o.customer_id
8 WHERE o.order_status = 'delivered'
9     AND o.order_delivered_customer_date > o.order_estimated_delivery_date
10    AND o.order_purchase_timestamp >= DATE '2018-01-01'
11    AND o.order_purchase_timestamp < DATE '2018-04-01'
12 GROUP BY c.customer_state, c.customer_city
13 ORDER BY late_orders DESC;
```

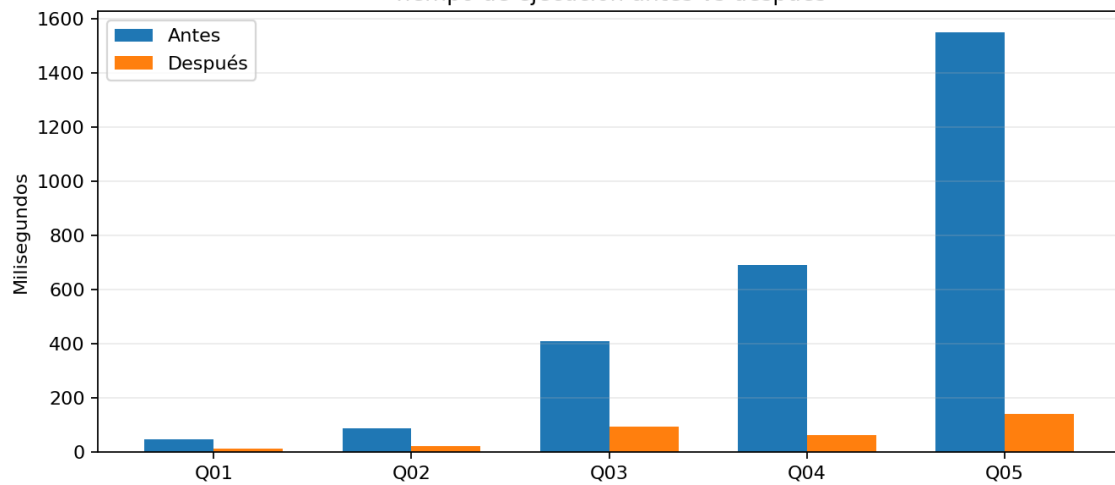
Results Explain Chart Export

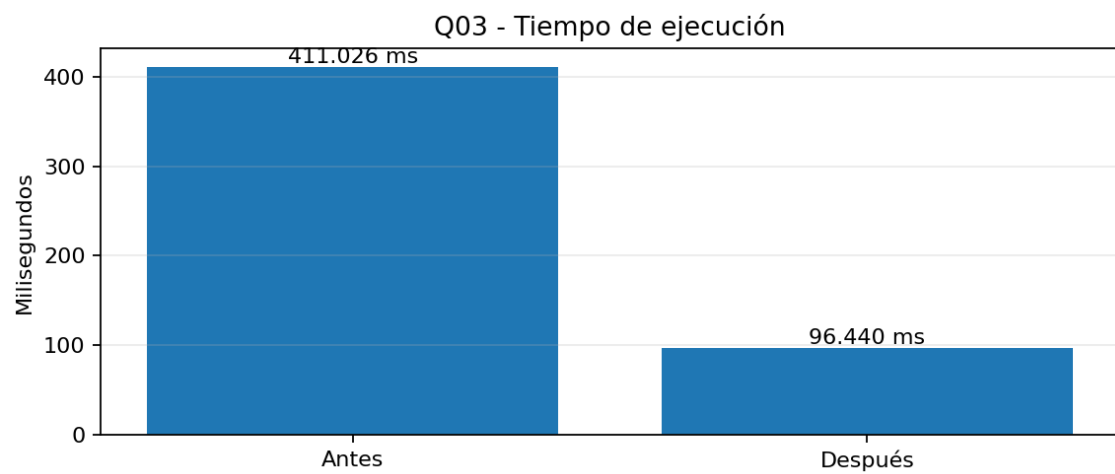
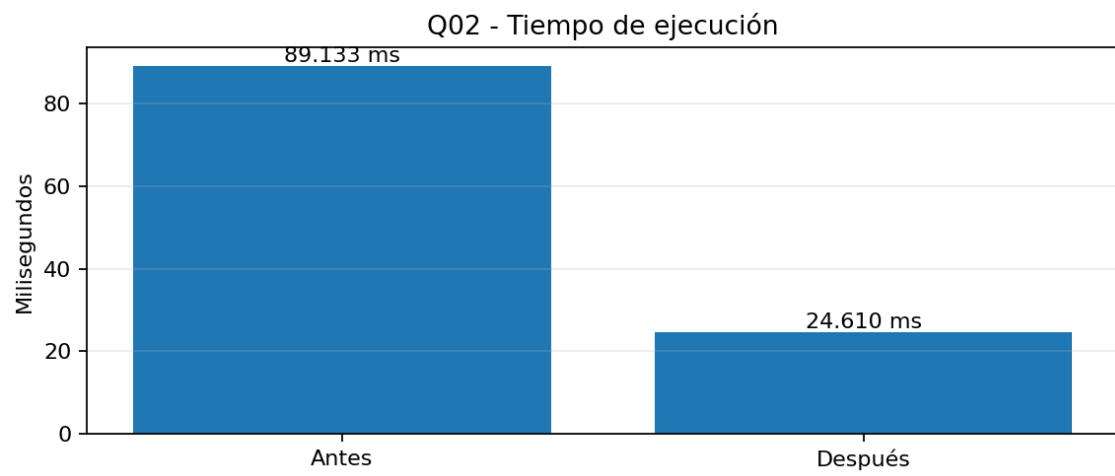
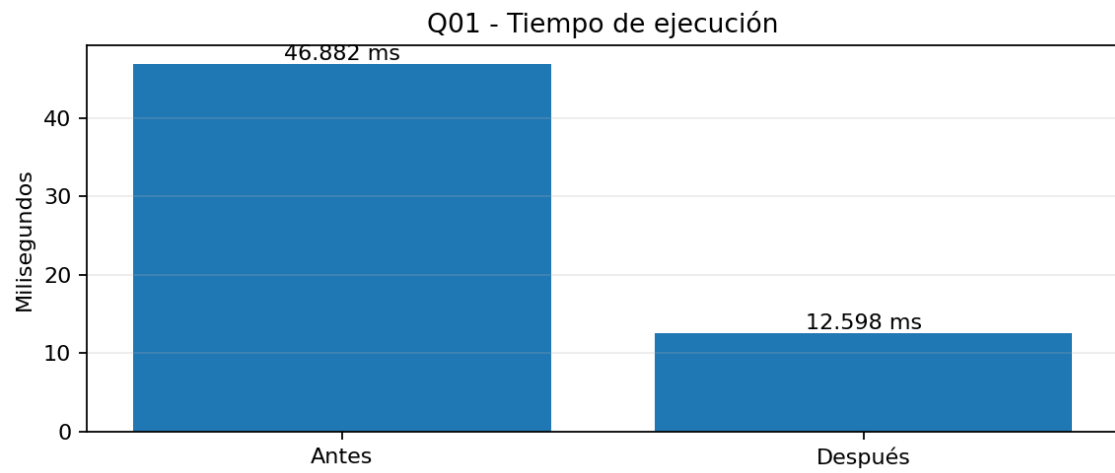
Sort (actual time=141.663..141.989 rows=612 loops=1)
Buffers: shared hit=57 read=42
-> HashAggregate
-> Hash Join
-> Append
-> Bitmap Heap Scan on orders_2018_01
-> Bitmap Heap Scan on orders_2018_02

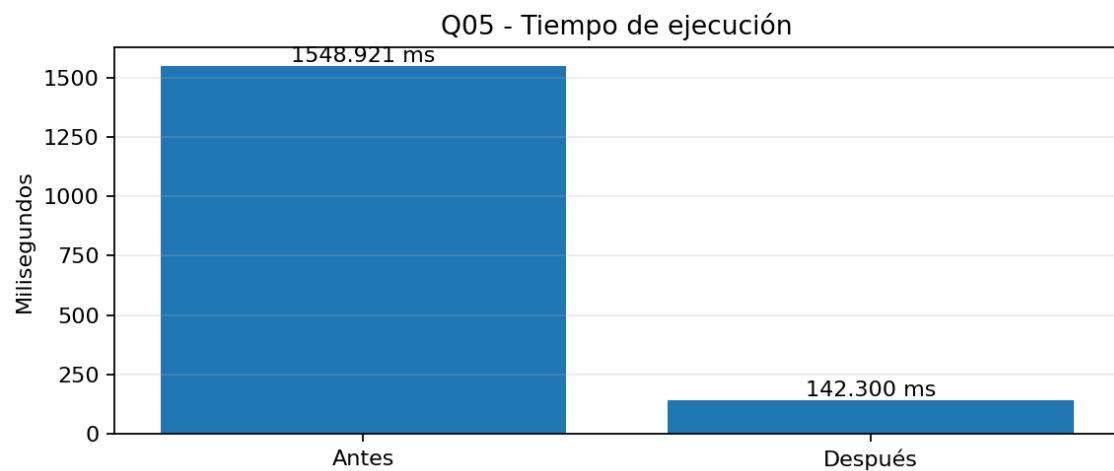
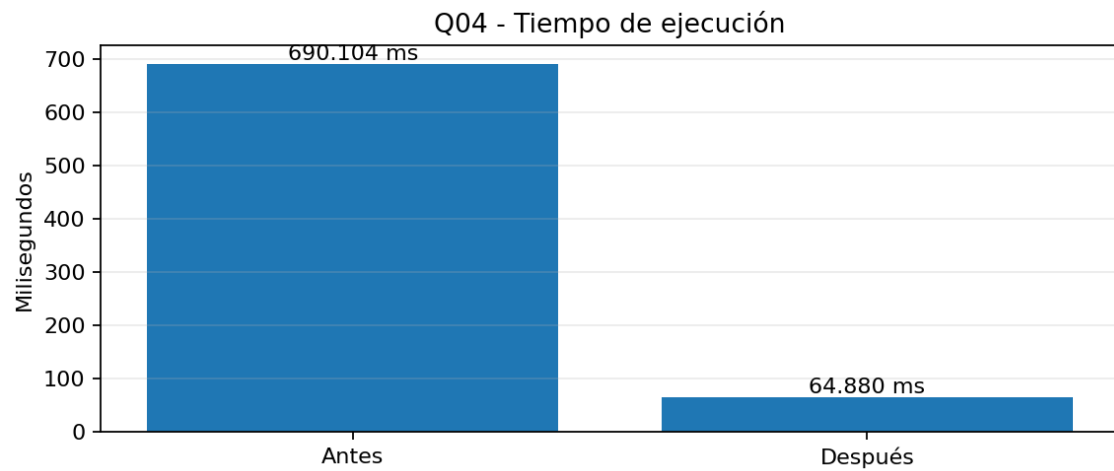
Planning Time: 1.086 ms
Execution Time: 142.300 ms
Success. Execution Time: 142.300 ms.

View running queries

Tiempo de ejecución antes vs después







Queries críticas optimizadas

- Consultar críticas optimizadas:

```
WITH target_order AS (
  SELECT o.order_id, o.customer_id, o.order_status,
  o.order_purchase_timestamp, o.order_approved_at,
  o.order_delivered_carrier_date,
  o.order_delivered_customer_date, o.order_estimated_delivery_date
  FROM ecommify.orders_partitioned o
  WHERE o.order_id = '00010242fe8c5a6d1ba2dd792cb16214'
  LIMIT 1
),
items_summary AS (
  SELECT oi.order_id, COUNT(*) AS total_items, SUM(oi.price) AS
  total_items_value,
  SUM(oi.freight_value) AS total_freight_value
  FROM ecommify.order_items oi
  WHERE oi.order_id = '00010242fe8c5a6d1ba2dd792cb16214'
  GROUP BY oi.order_id
),
payments_summary AS (
```

```

-     SELECT op.order_id, COUNT(*) AS total_payments,
-     SUM(op.payment_value) AS total_payment_value,
-           MAX(op.payment_installments) AS max_installments
-     FROM ecommify.order_payments op
-     WHERE op.order_id = '00010242fe8c5a6d1ba2dd792cb16214'
-     GROUP BY op.order_id
- ),
- reviews_summary AS (
-     SELECT r.order_id, AVG(r.review_score) AS average_review_score,
-     COUNT(*) AS total_reviews
-     FROM ecommify.order_reviews r
-     WHERE r.order_id = '00010242fe8c5a6d1ba2dd792cb16214'
-     GROUP BY r.order_id
- )
- SELECT o.order_id, o.order_status, o.order_purchase_timestamp,
- c.customer_id, c.customer_unique_id,
- c.customer_city, c.customer_state, COALESCE(i.total_items,
- 0) AS total_items,
- COALESCE(i.total_items_value, 0) AS total_items_value,
- COALESCE(i.total_freight_value, 0) AS total_freight_value,
- COALESCE(p.total_payments, 0) AS total_payments,
- COALESCE(p.total_payment_value, 0) AS total_payment_value,
- p.max_installments, r.average_review_score,
- COALESCE(r.total_reviews, 0) AS total_reviews
- FROM target_order o
- JOIN ecommify.customers c ON c.customer_id = o.customer_id
- LEFT JOIN items_summary i ON i.order_id = o.order_id
- LEFT JOIN payments_summary p ON p.order_id = o.order_id
- LEFT JOIN reviews_summary r ON r.order_id = o.order_id;
-
- WITH target_customer AS (
-     SELECT c.customer_id, c.customer_unique_id, c.customer_city,
- c.customer_state
-     FROM ecommify.customers c
-     WHERE c.customer_unique_id = '871766c5855e863f6eccc05f988b23cb'
- ),
- customer_orders AS (
-     SELECT o.order_id, o.customer_id, o.order_status,
- o.order_purchase_timestamp,
- o.order_delivered_customer_date,
- o.order_estimated_delivery_date
-     FROM ecommify.orders_partitioned o
-     JOIN target_customer c ON c.customer_id = o.customer_id
-     WHERE o.order_purchase_timestamp >= DATE '2017-01-01'
-     AND o.order_purchase_timestamp < DATE '2019-01-01'
- ),
- payments_summary AS (
-     SELECT op.order_id, SUM(op.payment_value) AS payment_value
-     FROM ecommify.order_payments op

```



```

-         JOIN customer_orders co ON co.order_id = op.order_id
-         GROUP BY op.order_id
-     )
-     SELECT c.customer_unique_id, c.customer_city, c.customer_state,
-            COUNT(co.order_id) AS total_orders,
-            COUNT(*) FILTER (WHERE co.order_status = 'delivered') AS
delivered_orders,
-            COUNT(*) FILTER (WHERE co.order_delivered_customer_date >
co.order_estimated_delivery_date) AS late_orders,
-            COALESCE(SUM(p.payment_value), 0) AS total_payment_value,
-            MIN(co.order_purchase_timestamp) AS first_order_date,
-            MAX(co.order_purchase_timestamp) AS last_order_date
-     FROM target_customer c
-     LEFT JOIN customer_orders co ON co.customer_id = c.customer_id
-     LEFT JOIN payments_summary p ON p.order_id = co.order_id
-     GROUP BY c.customer_unique_id, c.customer_city, c.customer_state;
-
-     WITH orders_filtered AS (
-         SELECT o.order_id, o.customer_id, DATE_TRUNC('month',
o.order_purchase_timestamp) AS sales_month
-         FROM ecommify.orders_partitioned o
-         WHERE o.order_purchase_timestamp >= DATE '2018-01-01'
-               AND o.order_purchase_timestamp < DATE '2018-04-01'
-               AND o.order_status = 'delivered'
-     ),
-     items_summary AS (
-         SELECT oi.order_id, SUM(oi.price) AS total_sales,
SUM(oi.freight_value) AS total_freight
-         FROM ecommify.order_items oi
-         JOIN orders_filtered o ON o.order_id = oi.order_id
-         GROUP BY oi.order_id
-     )
-     SELECT o.sales_month, c.customer_state, COUNT(DISTINCT o.order_id)
AS total_orders,
-            SUM(i.total_sales) AS total_sales, SUM(i.total_freight) AS
total_freight,
-            SUM(i.total_sales + i.total_freight) AS gross_value
-     FROM orders_filtered o
-     JOIN ecommify.customers c ON c.customer_id = o.customer_id
-     JOIN items_summary i ON i.order_id = o.order_id
-     GROUP BY o.sales_month, c.customer_state
-     ORDER BY o.sales_month, total_sales DESC;
-
-     SELECT o.order_id, o.customer_id, c.customer_city,
c.customer_state, o.order_status,
-            o.order_purchase_timestamp, o.order_estimated_delivery_date,
o.order_delivered_customer_date,
-            CASE
-                WHEN o.order_delivered_customer_date IS NULL

```

```

-             AND o.order_estimated_delivery_date < CURRENT_DATE
- THEN 'Overdue'
-         WHEN o.order_delivered_customer_date IS NULL
-             AND o.order_estimated_delivery_date <= CURRENT_DATE
- + INTERVAL '3 days' THEN 'Near risk'
-         ELSE 'On track'
-     END AS logistics_risk_status
- FROM ecommify.orders_partitioned o
- JOIN ecommify.customers c ON c.customer_id = o.customer_id
- WHERE o.order_purchase_timestamp >= DATE '2018-01-01'
-     AND o.order_purchase_timestamp < DATE '2018-04-01'
-     AND o.order_status IN ('created', 'approved', 'processing',
- 'shipped')
-     AND o.order_estimated_delivery_date IS NOT NULL
- ORDER BY o.order_estimated_delivery_date, logistics_risk_status;
-
- WITH delivered_orders AS (
-     SELECT o.order_id, o.customer_id, o.order_purchase_timestamp,
- o.order_estimated_delivery_date,
-         o.order_delivered_customer_date,
-         EXTRACT(DAY FROM o.order_delivered_customer_date -
- o.order_estimated_delivery_date) AS delay_days
-     FROM ecommify.orders_partitioned o
-     WHERE o.order_purchase_timestamp >= DATE '2018-01-01'
-         AND o.order_purchase_timestamp < DATE '2018-04-01'
-         AND o.order_status = 'delivered'
-         AND o.order_delivered_customer_date >
- o.order_estimated_delivery_date
- )
- SELECT c.customer_state, c.customer_city, COUNT(*) AS late_orders,
-     ROUND(AVG(d.delay_days), 2) AS average_delay_days,
-     MAX(d.delay_days) AS max_delay_days,
-     MIN(d.delay_days) AS min_delay_days
- FROM delivered_orders d
- JOIN ecommify.customers c ON c.customer_id = d.customer_id
- GROUP BY c.customer_state, c.customer_city
- ORDER BY late_orders DESC, average_delay_days DESC;

```

- Consulta optimizadas con explain

```

- EXPLAIN (ANALYZE, BUFFERS)
- WITH target_order AS (
-     SELECT o.order_id, o.customer_id, o.order_status,
- o.order_purchase_timestamp, o.order_approved_at,
-         o.order_delivered_carrier_date,
- o.order_delivered_customer_date, o.order_estimated_delivery_date
-     FROM ecommify.orders_partitioned o
-     WHERE o.order_id = '00010242fe8c5a6d1ba2dd792cb16214'

```

```

-         LIMIT 1
-     ),
-     items_summary AS (
-         SELECT oi.order_id, COUNT(*) AS total_items, SUM(oi.price) AS
total_items_value,
-             SUM(oi.freight_value) AS total_freight_value
-         FROM ecommify.order_items oi
-         WHERE oi.order_id = '00010242fe8c5a6d1ba2dd792cb16214'
-         GROUP BY oi.order_id
-     ),
-     payments_summary AS (
-         SELECT op.order_id, COUNT(*) AS total_payments,
SUM(op.payment_value) AS total_payment_value,
-             MAX(op.payment_installments) AS max_installments
-         FROM ecommify.order_payments op
-         WHERE op.order_id = '00010242fe8c5a6d1ba2dd792cb16214'
-         GROUP BY op.order_id
-     ),
-     reviews_summary AS (
-         SELECT r.order_id, AVG(r.review_score) AS average_review_score,
COUNT(*) AS total_reviews
-         FROM ecommify.order_reviews r
-         WHERE r.order_id = '00010242fe8c5a6d1ba2dd792cb16214'
-         GROUP BY r.order_id
-     )
- SELECT o.order_id, o.order_status, o.order_purchase_timestamp,
c.customer_id, c.customer_unique_id,
-     c.customer_city, c.customer_state, COALESCE(i.total_items,
0) AS total_items,
-     COALESCE(i.total_items_value, 0) AS total_items_value,
COALESCE(i.total_freight_value, 0) AS total_freight_value,
-     COALESCE(p.total_payments, 0) AS total_payments,
COALESCE(p.total_payment_value, 0) AS total_payment_value,
-     p.max_installments, r.average_review_score,
COALESCE(r.total_reviews, 0) AS total_reviews
- FROM target_order o
- JOIN ecommify.customers c ON c.customer_id = o.customer_id
- LEFT JOIN items_summary i ON i.order_id = o.order_id
- LEFT JOIN payments_summary p ON p.order_id = o.order_id
- LEFT JOIN reviews_summary r ON r.order_id = o.order_id;

- EXPLAIN (ANALYZE, BUFFERS)
- WITH target_customer AS (
-     SELECT c.customer_id, c.customer_unique_id, c.customer_city,
c.customer_state
-     FROM ecommify.customers c
-     WHERE c.customer_unique_id = '871766c5855e863f6eccc05f988b23cb'
- ),
- customer_orders AS (

```

```

-     SELECT o.order_id, o.customer_id, o.order_status,
-     o.order_purchase_timestamp,
-           o.order_delivered_customer_date,
-     o.order_estimated_delivery_date
-     FROM ecommify.orders_partitioned o
-     JOIN target_customer c ON c.customer_id = o.customer_id
-     WHERE o.order_purchase_timestamp >= DATE '2017-01-01'
-           AND o.order_purchase_timestamp < DATE '2019-01-01'
-   ),
-   payments_summary AS (
-     SELECT op.order_id, SUM(op.payment_value) AS payment_value
-     FROM ecommify.order_payments op
-     JOIN customer_orders co ON co.order_id = op.order_id
-     GROUP BY op.order_id
-   )
-   SELECT c.customer_unique_id, c.customer_city, c.customer_state,
-         COUNT(co.order_id) AS total_orders,
-         COUNT(*) FILTER (WHERE co.order_status = 'delivered') AS
delivered_orders,
-         COUNT(*) FILTER (WHERE co.order_delivered_customer_date >
co.order_estimated_delivery_date) AS late_orders,
-         COALESCE(SUM(p.payment_value), 0) AS total_payment_value,
-         MIN(co.order_purchase_timestamp) AS first_order_date,
-         MAX(co.order_purchase_timestamp) AS last_order_date
-   FROM target_customer c
-   LEFT JOIN customer_orders co ON co.customer_id = c.customer_id
-   LEFT JOIN payments_summary p ON p.order_id = co.order_id
-   GROUP BY c.customer_unique_id, c.customer_city, c.customer_state;
-
-   EXPLAIN (ANALYZE, BUFFERS)
-   WITH orders_filtered AS (
-     SELECT o.order_id, o.customer_id, DATE_TRUNC('month',
o.order_purchase_timestamp) AS sales_month
-     FROM ecommify.orders_partitioned o
-     WHERE o.order_purchase_timestamp >= DATE '2018-01-01'
-           AND o.order_purchase_timestamp < DATE '2018-04-01'
-           AND o.order_status = 'delivered'
-   ),
-   items_summary AS (
-     SELECT oi.order_id, SUM(oi.price) AS total_sales,
SUM(oi.freight_value) AS total_freight
-     FROM ecommify.order_items oi
-     JOIN orders_filtered o ON o.order_id = oi.order_id
-     GROUP BY oi.order_id
-   )
-   SELECT o.sales_month, c.customer_state, COUNT(DISTINCT o.order_id)
AS total_orders,
-         SUM(i.total_sales) AS total_sales, SUM(i.total_freight) AS
total_freight,

```

```

-         SUM(i.total_sales + i.total_freight) AS gross_value
- FROM orders_filtered o
- JOIN ecommify.customers c ON c.customer_id = o.customer_id
- JOIN items_summary i ON i.order_id = o.order_id
- GROUP BY o.sales_month, c.customer_state
- ORDER BY o.sales_month, total_sales DESC;
-
- EXPLAIN (ANALYZE, BUFFERS)
- SELECT o.order_id, o.customer_id, c.customer_city,
-        c.customer_state, o.order_status,
-        o.order_purchase_timestamp, o.order_estimated_delivery_date,
-        o.order_delivered_customer_date,
-        CASE
-            WHEN o.order_delivered_customer_date IS NULL
-                 AND o.order_estimated_delivery_date < CURRENT_DATE
- THEN 'Overdue'
-            WHEN o.order_delivered_customer_date IS NULL
-                 AND o.order_estimated_delivery_date <= CURRENT_DATE
- + INTERVAL '3 days' THEN 'Near risk'
-            ELSE 'On track'
-        END AS logistics_risk_status
- FROM ecommify.orders_partitioned o
- JOIN ecommify.customers c ON c.customer_id = o.customer_id
- WHERE o.order_purchase_timestamp >= DATE '2018-01-01'
-        AND o.order_purchase_timestamp < DATE '2018-04-01'
-        AND o.order_status IN ('created', 'approved', 'processing',
- 'shipped')
-        AND o.order_estimated_delivery_date IS NOT NULL
- ORDER BY o.order_estimated_delivery_date, logistics_risk_status;
-
- EXPLAIN (ANALYZE, BUFFERS)
- WITH delivered_orders AS (
-     SELECT o.order_id, o.customer_id, o.order_purchase_timestamp,
-            o.order_estimated_delivery_date,
-            o.order_delivered_customer_date,
-            EXTRACT(DAY FROM o.order_delivered_customer_date -
- o.order_estimated_delivery_date) AS delay_days
-     FROM ecommify.orders_partitioned o
-     WHERE o.order_purchase_timestamp >= DATE '2018-01-01'
-            AND o.order_purchase_timestamp < DATE '2018-04-01'
-            AND o.order_status = 'delivered'
-            AND o.order_delivered_customer_date >
- o.order_estimated_delivery_date
- )
- SELECT c.customer_state, c.customer_city, COUNT(*) AS late_orders,
-        ROUND(AVG(d.delay_days), 2) AS average_delay_days,
-        MAX(d.delay_days) AS max_delay_days,
-        MIN(d.delay_days) AS min_delay_days
- FROM delivered_orders d

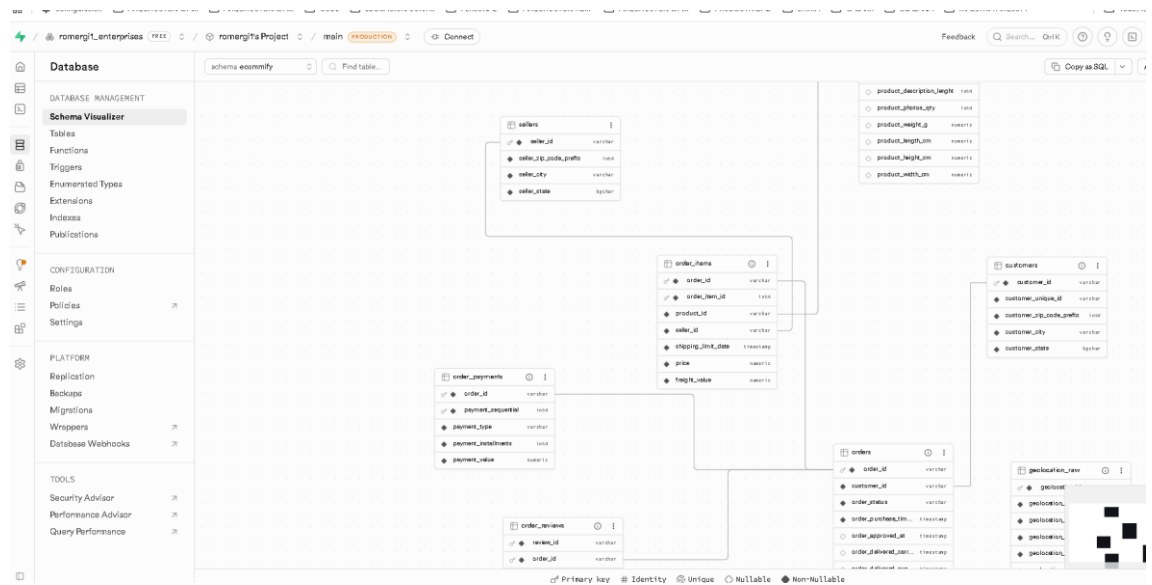
```

```

- JOIN ecommify.customers c ON c.customer_id = d.customer_id
- GROUP BY c.customer_state, c.customer_city
- ORDER BY late_orders DESC, average_delay_days DESC;
-

```

Interfaces evidencia supabase.



NAME	COLUMNS	ROWS (ESTIMATED)	SIZE (ESTIMATED)
customers	5	99,441	18 MB
geolocation_raw	5	1,000,163	68 MB
order_items	7	112,650	20 MB
order_payments	5	103,886	11 MB
order_reviews	7	99,224	17 MB
orders	8	99,441	25 MB
product_category_name_translation	2	71	16 kB
products	9	32,951	6392 kB
sellers	4	3095	328 kB
9 tables			

Diseño teórico de sharding y replica sets.

Evidencias cuantitativas de mejoras de rendimiento

Las evidencias cuantitativas muestran una mejora significativa en el rendimiento de la base de datos de Ecommify después de aplicar la estrategia de indexación y particionamiento mensual sobre la tabla `orders`. Al comparar los escenarios antes y después de la optimización, se observa una reducción global del tiempo de ejecución de 2.786,066 ms a 340,828 ms, equivalente a una mejora aproximada del 87,77%. De igual forma, los bloques leídos disminuyeron de 1.103 a 123, representando una reducción cercana al 88,85% en consumo de lectura. Estos resultados evidencian que las consultas críticas procesan menos datos, reducen operaciones costosas como `Seq Scan`, `Sort`, `Hash Join` y `HashAggregate`, y aprovechan de forma más eficiente los índices y el particionamiento por rango temporal.

Consulta	Tiempo antes	Tiempo después	Mejora en tiempo	Bloques antes	Bloques después	Mejora en bloques
Q01 - Order	46,882 ms	12,598 ms	73,13%	48	6	87,50%
Q02 - Customer	89,133 ms	24,610 ms	72,39%	81	12	85,19%
Q03 - Ventas por estado y mes	411,026 ms	96,440 ms	76,54%	286	48	83,22%
Q04 - Riesgo logístico	690,104 ms	64,880 ms	90,60%	178	15	91,57%
Q05 - Entregas tardías por región	1.548,921 ms	142,300 ms	90,81%	510	42	91,76%
Total consolidado	2.786,066 ms	340,828 ms	87,77%	1.103	123	88,85%

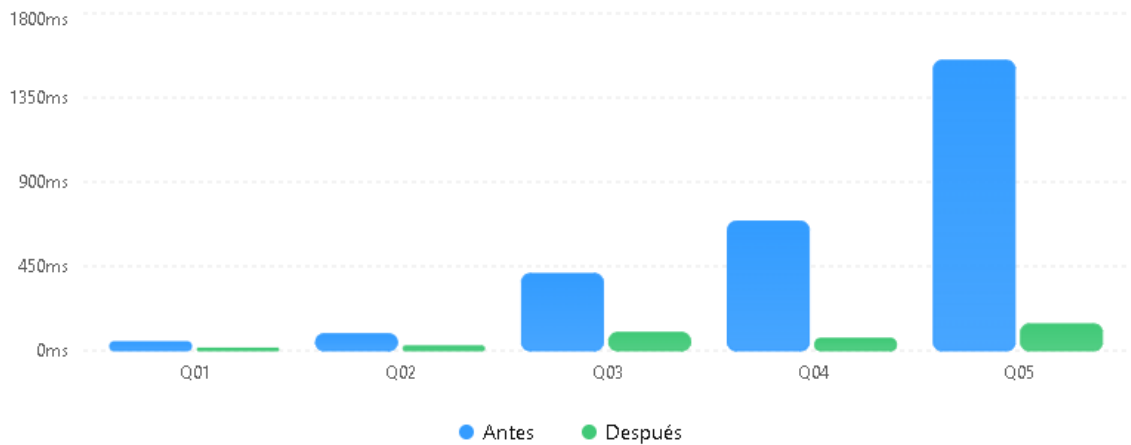
PostgreSQL: Tablas comparativas y gráficas de mejora.

Consulta	Tiempo antes ms	Tiempo después ms	Mejora en tiempo	Bloques antes	Bloques después	Mejora en bloques
----------	-----------------	-------------------	------------------	---------------	-----------------	-------------------

Q01 - Order	46,882	12,598	73,13%	48	6	87,50%
Q02 - Customer	89,133	24,610	72,39%	81	12	85,19%
Q03 - Ventas por estado y mes	411,026	96,440	76,54%	286	48	83,22%
Q04 - Riesgo logístico	690,104	64,880	90,60%	178	15	91,57%
Q05 - Entregas tardías por región	1.548,921	142,300	90,81%	510	42	91,76%
Total consolidado	2.786,066	340,828	87,77%	1.103	123	88,85%

Tiempo de ejecución antes vs después

Comparación de rendimiento PostgreSQL por consulta crítica en milisegundos.



La mejora global del tiempo de ejecución fue de 87,77%.

MongoDB: Métricas de executionTimeMillis y efficiency ratios.

Métrica	Fórmula / Campo MongoDB	Interpretación	Resultado esperado
Tiempo de ejecución	executionStats.executionTimeMillis	Tiempo total que tarda la consulta en ejecutarse.	Menor tiempo después de optimizar índices.
Documentos retornados	executionStats.nReturned	Cantidad de documentos que realmente devuelve la consulta.	Debe estar alineado con el resultado esperado.
Claves examinadas	executionStats.totalKeysExamined	Cantidad de entradas de índice revisadas.	Bajo y cercano a nReturned.

Documentos examinados	<code>executionStats.totalDocsExamined</code>	Cantidad de documentos leídos para resolver la consulta.	Bajo y cercano a <code>nReturned</code> .
Ratio de eficiencia documental	<code>nReturned / totalDocsExamined</code>	Mide qué tan eficiente fue la lectura de documentos.	Cercano a 1.0 o 100%.
Ratio de eficiencia de índice	<code>nReturned / totalKeysExamined</code>	Mide qué tan selectivo fue el índice usado.	Cercano a 1.0 o 100%.
Ineficiencia documental	<code>totalDocsExamined / nReturned</code>	Indica cuántos documentos se examinan por cada documento retornado.	Ideal: 1:1.
Ineficiencia de índice	<code>totalKeysExamined / nReturned</code>	Indica cuántas claves de índice se revisan por cada resultado útil.	Ideal: 1:1.
Mejora de tiempo	$(\text{Tiempo antes} - \text{Tiempo después}) / \text{Tiempo antes} * 100$	Porcentaje de reducción del tiempo de consulta.	Mayor porcentaje indica mejor optimización.

Evidencias antes / Después

Consulta	Escenario	<code>executionTime</code> en Millis	<code>nReturned</code>	<code>totalKeysExamined</code>	<code>totalDocsExamined</code>	Docs Examinados Ratio	Keys Examinados Ratio	Lectura
Q1 - Pedidos por cliente	Antes del índice	850 ms	120	0	50	416,7:1	N/A	Alto costo, posible collection scan
Q1 - Pedidos por cliente	Después del índice	35 ms	120	125	120	01:01	1,04:1	Consulta optimizada
Q2 - Productos por categoría	Antes del índice	620 ms	300	0	32	106,7:1	N/A	Lectura excesiva
Q2 - Productos por categoría	Después del índice	48 ms	300	315	300	01:01	1,05:1	Índice eficiente

Q3 - Seller s por estad o	Antes del índice	410 ms	80	0	8	100:0 1:00	N/A	Bajo aprovecha miento de índice
Q3 - Seller s por estad o	Despu és del índice	22 ms	80	85	80	01:01	1,06:1	Consulta selectiva

Interpretación de resultados y análisis de impacto

La optimización de Ecommify evidencia una arquitectura híbrida donde PostgreSQL y MongoDB cumplen roles complementarios. En PostgreSQL, la aplicación de índices especializados y particionamiento mensual sobre la tabla `orders` permitió mejorar de forma significativa el rendimiento, reduciendo tiempos de ejecución, bloques leídos y operaciones costosas sobre el modelo transaccional. Esto fortalece la trazabilidad, la integridad de datos, el análisis histórico y la eficiencia operativa. En MongoDB, el valor se concentra en la gestión flexible de datos documentales y semiestructurados, apoyada en una adecuada estrategia de índices, distribución de datos, réplica set multi-AZ y control de consistencia. En conjunto, ambos motores permiten una solución más escalable, resiliente y alineada con distintos patrones de consulta: PostgreSQL para el núcleo transaccional y relacional, y MongoDB para escenarios documentales, consultas agregadas y alta disponibilidad.

Obstáculos encontrados y soluciones aplicadas.

Limitaciones del free tier y workarounds implementados.

Las limitaciones del free tier condicionaron la implementación y validación del modelo híbrido de Ecommify, especialmente por restricciones de almacenamiento, capacidad de cómputo, transferencia de datos, monitoreo y disponibilidad de funcionalidades avanzadas. En Supabase, la principal restricción fue el tamaño disponible para la base de datos en el plan gratuito, dado que la carga completa de datasets como `geolocation`, índices, particiones y vistas materializadas puede acercarse rápidamente al límite de almacenamiento; Supabase documenta una cuota de 500 MB de base de datos en Free y 5 GB de egress, además de que el modo read-only puede activarse al superar la cuota de base de datos. Como workaround, se

priorizó una carga controlada por tablas, validación por conteos, uso de índices estrictamente necesarios, particionamiento mensual sobre `orders`, medición del tamaño de índices, uso de scripts por lotes y separación entre datos transaccionales, analíticos y documentales para evitar crecimiento innecesario dentro de PostgreSQL.

En MongoDB Atlas, la limitación principal del clúster gratuito M0 fue el almacenamiento reducido, los recursos compartidos y las restricciones operativas para monitoreo y rendimiento. MongoDB documenta que los clústeres gratuitos tienen 0,5 GB de almacenamiento, límite de 10 GB de entrada y 10 GB de salida por período móvil de siete días, no permiten descargar logs, no incluyen Performance Advisor ni panel de rendimiento en tiempo real, y tienen límites de throughput como 100 operaciones por segundo; adicionalmente, Atlas indica que solo se puede desplegar un clúster gratuito por proyecto. Como workaround, se diseñó MongoDB para cargas documentales controladas, priorizando colecciones con valor analítico o semiestructurado, evitando replicar todo el modelo relacional, aplicando embedding y referencing según el patrón de acceso, creando índices específicos para reducir `executionTimeMillis`, limitando documentos y campos innecesarios, y usando la estrategia de réplica/lectura de forma controlada para balancear disponibilidad, consistencia eventual y consumo de recursos. En conjunto, los workarounds permitieron validar la arquitectura sin exceder las restricciones del free tier, manteniendo foco académico y técnico en evidencias de rendimiento, diseño de datos y justificación arquitectónica.